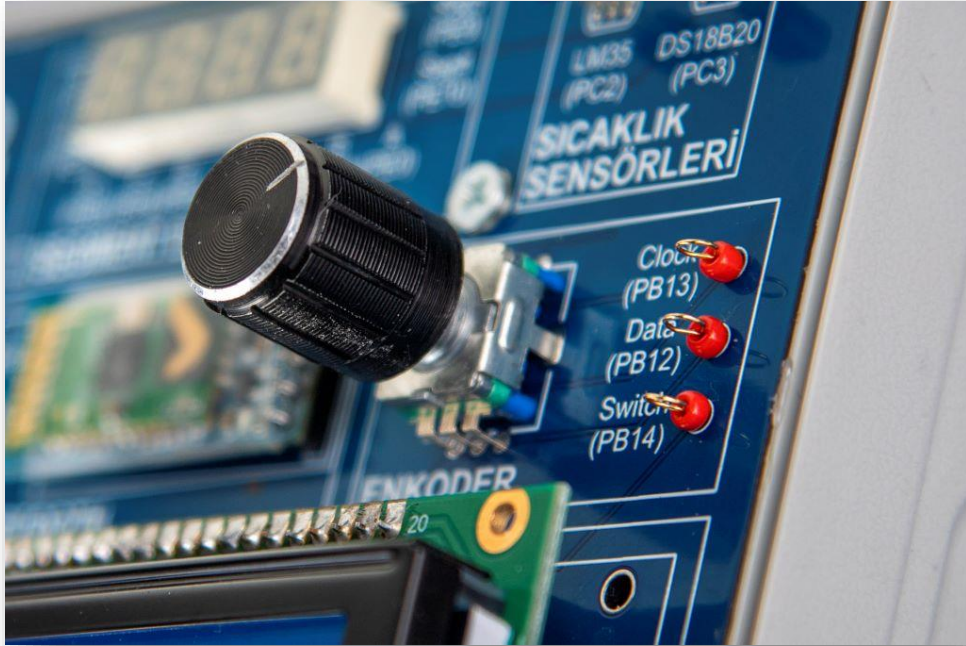




**SİVAS CUMHURİYET
ÜNİVERSİTESİ**
BİLGİSAYAR MÜHENDİSLİĞİ
BÖLÜMÜ
GÖMÜLÜ SİSTEMLER DERSİ

Gömülü Sistemler Deney 3: Harici Kesmeler (External Interrupts - EXTI)

Rotary Encoder Okuma, Yön Tespiti ve Menü Kontrolü Uygulaması



Dersin Hocası: Doç. Dr. Ahmet Gürkan YÜKSEK

Dersin Asistanı: Arş. Gör. Ahmet Utku ELİK

Öğrenci Adı Soyadı – Numarası:

1. Deneyin Amacı

Bu deneyde, STM32F407 mikrodnetleyicisinin **EXTI (External Interrupt)** birimini kullanarak dış dünyadan gelen asenkron sinyallerin (Enkoder hareketi) işlemciyi meşgul etmeden (Interrupt yöntemiyle) algılanması amaçlanmaktadır. Ayrıca mekanik bir sensör olan **Rotary Encoder**'ın çalışma prensibi ve yazılımsal olarak okunması (Decoding) öğrenilecektir.

2. Kesme (Interrupt) Nedir?

Mikrodnetleyici normal şartlarda kodları satır satır (yukarıdan aşağıya) işler. Ancak acil bir durum olduğunda (bir butona basılması, verinin gelmesi vb.) işlemcinin o anki işini bırakıp bu acil durumla ilgilenmesi gerekebilir. Bu mekanizmaya **Interrupt (Kesme)** denir.

2.1. Polling (Sorgulama) vs. Interrupt (Kesme) Giriş birimlerini okumanın iki temel yolu vardır:

1. **Polling (Sorgulama):** İşlemci sürekli "Butona basıldı mı?" diye kontrol eder. Bu yöntem işlemciyi yorar ve gereksiz zaman harcar.
2. **Interrupt (Kesme):** İşlemci kendi işine bakar. Butona basıldığında donanım işlemciyi dırter ("Hey! Bana basıldı!"). İşlemci işini bırakır, butona ait kodu çalıştırır ve kaldığı yerden devam eder.

Polling ve Interrupt Mekanizmalarının Karşılaştırılması

• Polling (Sorgulama Yöntemi):

Sistem, belirli aralıklarla veya sürekli olarak bir donanım durumunu kontrol eder. Bu yaklaşımda işlemci, ilgili olayın gerçekleşip gerçekleşmediğini aktif şekilde denetlediği için işlem süresinin önemli bir kısmı bu kontrole ayrılır. Polling yöntemi, basit yapısı nedeniyle tercih edilebilse de işlemci verimliliğini düşürür ve gereksiz kaynak kullanımına yol açabilir.

• Interrupt (Kesme Mekanizması):

Sistem, normal işlemlerine devam ederken donanım tarafından oluşturulan bir kesme sinyaliyle bilgilendirilir. Olay gerçekleştiğinde işlemci mevcut işlemini geçici olarak durdurur ve ilgili **Kesme Alt Programını (ISR – Interrupt Service Routine)** çalıştırır. Kesme işlemi tamamlandıktan sonra sistem, kaldığı yerden normal akışına devam eder. Bu yöntem, işlemci kaynaklarının daha verimli kullanılmasını sağlar ve gerçek zamanlı sistemlerde yaygın olarak tercih edilir.

3. Rotary Encoder (Döner Kodlayıcı) Çalışma Prensibi

Rotary Encoder, sonsuz tur dönebilen ve dönüş yönüne göre dijital sinyal üreten bir elektromekanik elemandır. İçerisinde A ve B (bizim setimizde CLK ve DT) olmak üzere iki adet çıkış pini bulunur.

- **Saat Yönü (CW):** A sinyali, B sinyalinden önce değişir.
- **Saat Yönü Ters (CCW):** B sinyali, A sinyalinden önce değişir. Biz bu deneyde A (CLK) pinindeki değişimi **Kesme (Interrupt)** olarak yakalayıp, o andaki B (DT) pininin durumuna bakarak yönü tayin edeceğiz.

4. Kullanılacak Donanım ve Pin Tablosu

ArmApp18 seti üzerindeki Rotary Encoder ve Slide Switch modülleri kullanılacaktır.

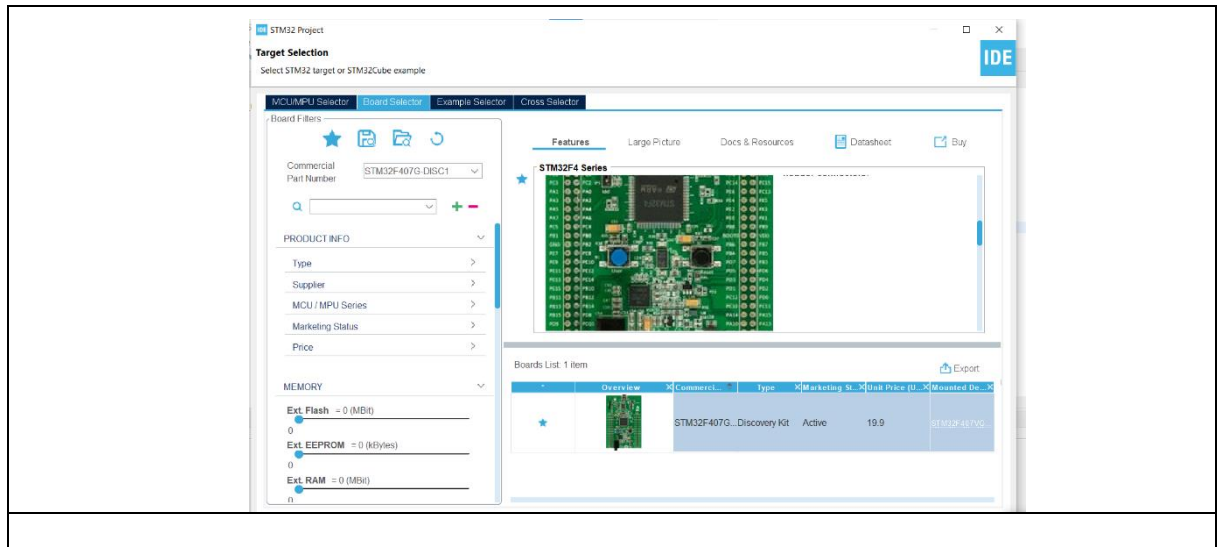
Bileşen	Pin Kodu	Mod (Mode)	Açıklama
Encoder CLK	PB13	GPIO_EXTI13	Enkoder Saat Sinyali (Kesme Kaynağı)
Encoder DT	PB12	GPIO_Input	Enkoder Data Sinyali (Yön Bilgisi)
Switch 6 (SW6)	PD10	GPIO_Input	Ekran Modu Seçimi (Görüntüleme)
LCD RS	PD2	GPIO_Output	LCD Register Select Pini
LCD EN	PD3	GPIO_Output	LCD Enable (Yetkilendirme) Pini
LCD D4	PC4	GPIO_Output	LCD Data Hattı 4
LCD D5	PC13	GPIO_Output	LCD Data Hattı 5
LCD D6	PC14	GPIO_Output	LCD Data Hattı 6
LCD D7	PC15	GPIO_Output	LCD Data Hattı 7

5. STM32CubeIDE Konfigürasyonu (CubeMX Ayarları)

Bu deneyde, işlemcinin sürekli pin kontrolü yapması yerine, donanımın işlemciyi uyarması (Interrupt) sağlanacaktır. Bunun için **EXTI (External Interrupt)** hattı ve **NVIC (Nested Vectored Interrupt Controller)** yapılandırılmalıdır.

Adım 1: Proje Oluşturma

1. STM32CubeIDE'de File > New > STM32 Project diyerek yeni proje başlatın.

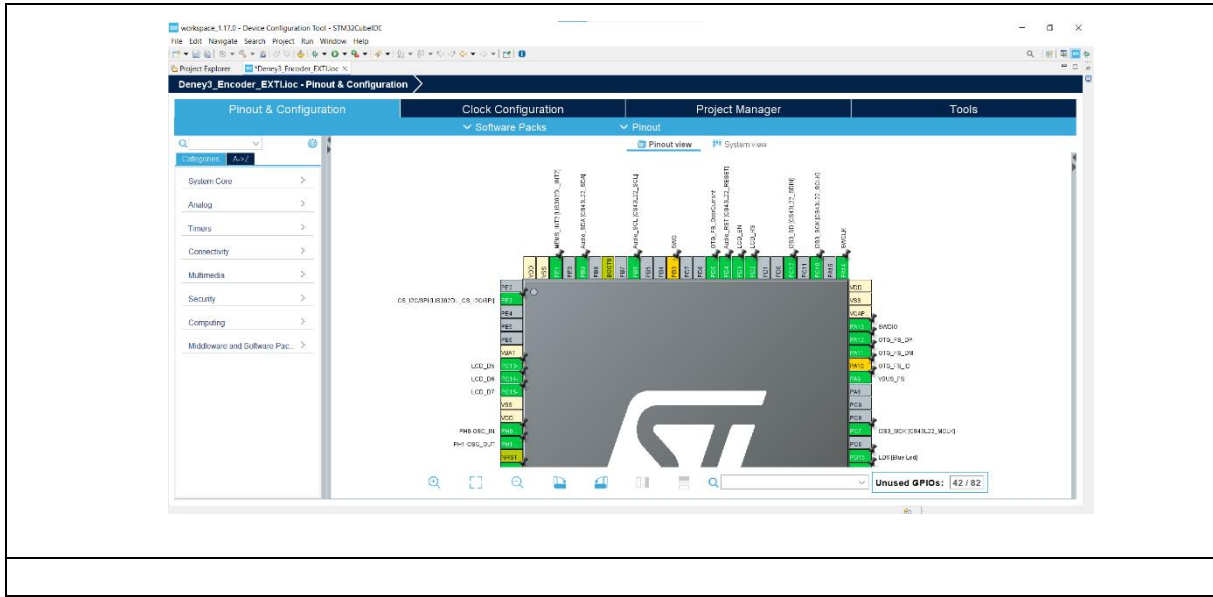


2. İşlemci olarak **STM32F407VG** seçin ve projeye "**Deney3_Encoder_EXTI**" ismini verin.

Adım 2: Pin Atamaları ve Etiketleme

Sağ taraftaki **Pinout View** ekranında aşağıdaki pinleri bulun, modlarını seçin ve sağ tıklayıp Enter User Label diyerek etiketlerini girin.

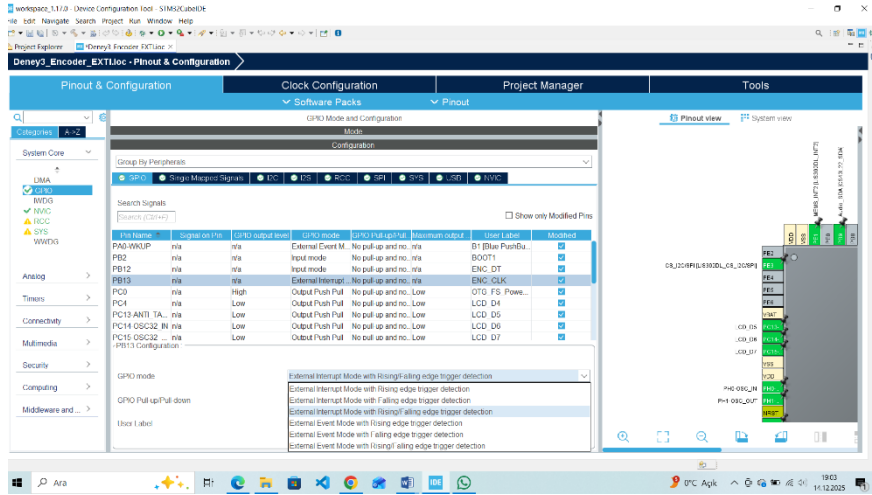
- **Rotary Encoder Ayarları:**
 - **PB13** -> GPIO_EXTI13 seçin. **Etiket:** ENC_CLK (Bu pin Kesme kaynağıdır).
 - **PB12** -> GPIO_Input seçin. **Etiket:** ENC_DT (Bu pin Yön verisidir).
- **Switch Ayarı:**
 - **PD10** -> GPIO_Input seçin. **Etiket:** SW6.
- **LCD Ekran Pinleri (Çıkış):**
 - PD2 -> **GPIO_Output** (Etiket: LCD_RS)
 - PD3 -> **GPIO_Output** (Etiket: LCD_EN)
 - PC4 -> **GPIO_Output** (Etiket: LCD_D4)
 - PC13 -> **GPIO_Output** (Etiket: LCD_D5)
 - PC14 -> **GPIO_Output** (Etiket: LCD_D6)
 - PC15 -> **GPIO_Output** (Etiket: LCD_D7)



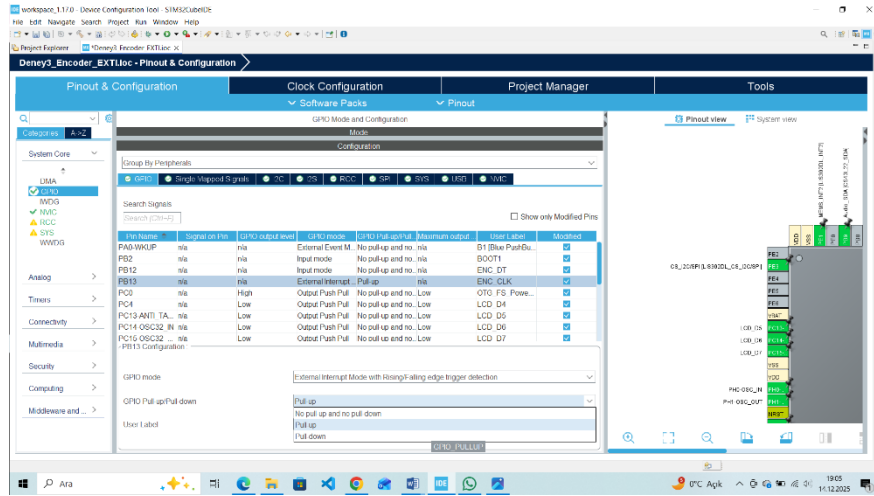
Adım 3: EXTI (Kesme) Modu Detay Ayarları

Kesmenin nasıl tetikleneceğini (döğmeye basınca mı, çekince mi?) belirlememiz gerekir.

1. Soldaki menüden **System Core > GPIO** sekmesine tıklayın.
2. Ortadaki listeden **PB13 (ENC_CLK)** satırına tıklayın.
3. Aşağıdaki **Configuration** penceresinde şu ayarları yapın:
 - **GPIO Mode:** External Interrupt Mode with Rising/Falling edge trigger detection.



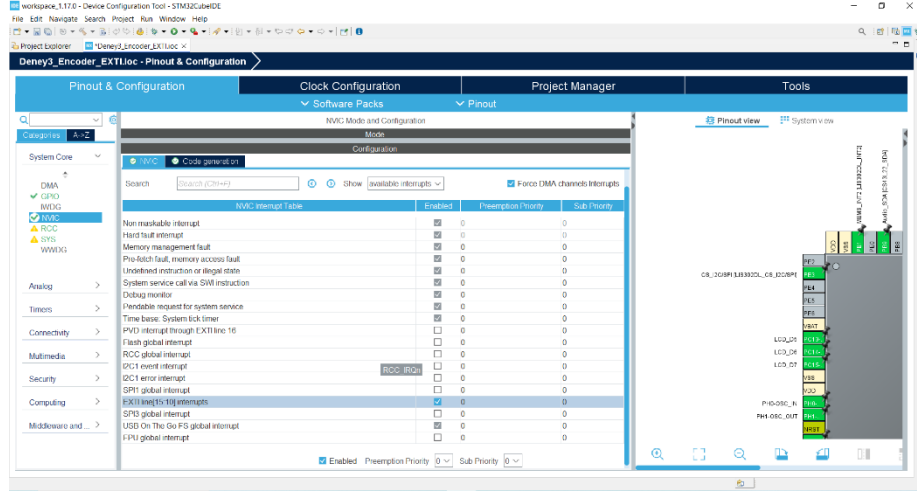
- *Neden?* Enkoderin her hareketini (hem 0'dan 1'e hem 1'den 0'a geçişi) kaçırmadan yakalamak için.
- **GPIO Pull-up/Pull-down:** Pull-up.
 - *Neden?* Sinyal boşta kaldığında kararlı bir "1" seviyesinde durması için.



Adım 4: NVIC (Kesme Yöneticisi) Aktivasyonu

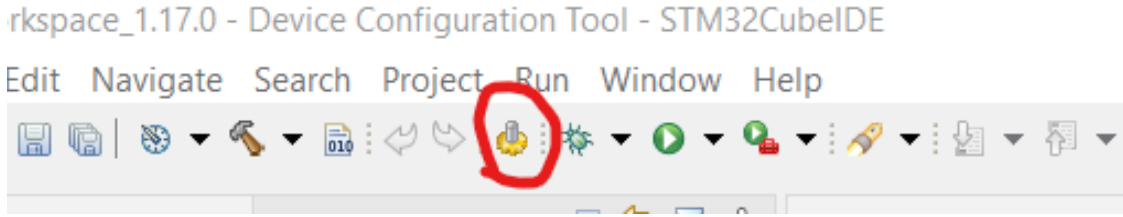
Donanımı ayarladık ama işlemciye "Bu kesmeyi dinle" dememiz lazım.

1. Soldaki menüden **System Core** > **NVIC** sekmesine tıklayın.
2. Listede **EXTI line[15:10] interrupts** satırını bulun.
3. **Enabled** sütunundaki kutucuğu işaretleyin (Tık atın).
 - *Not:* PB13 pini, 10 ile 15 arasındaki hatlara bağlı olduğu için bu satırı seçiyoruz.



Adım 5: Kod Üretimi

- Üst menüdeki **Sarı Dişli (Device Configuration Tool Code Generation)** ikonuna basarak main.c dosyasını oluşturun.



6. Yazılım ve Algoritma

6.1. LCD Kütüphanesinin Oluşturulması Bu deneyde kullanılan 16x02 LCD ekranın kontrolü için standart HAL kütüphanesinde hazır fonksiyonlar bulunmamaktadır. Bu nedenle lcd.h ve lcd.c sürücü dosyalarının projeye manuel olarak eklenmesi gerekmektedir.

Adım 1: lcd.h Dosyasının Oluşturulması Proje ağacındaki **Core/Inc** klasörüne sağ tıklayıp New > File diyerek **lcd.h** dosyasını oluşturun ve aşağıdaki kodları yazın:

```
/* lcd.h */
#ifndef INC_LCD_H_
#define INC_LCD_H_

#include "stm32f4xx_hal.h"

// Fonksiyon Prototipleri
void LCD_Init(void); // LCD Başlatma
void LCD_SendCommand(uint8_t cmd); // Komut Gönderme
void LCD_SendData(uint8_t data); // Veri Gönderme
void LCD_Clear(void); // Ekran Temizleme
void LCD_SetCursor(uint8_t row, uint8_t col); // İmleç Konumlandırma
void LCD_Print(char *str); // Metin Yazdırma

#endif /* INC_LCD_H_ */
```

Adım 2: lcd.c Dosyasının Oluşturulması Proje ağacındaki **Core/Src** klasörüne sağ tıklayıp New > File diyerek **lcd.c** dosyasını oluşturun ve aşağıdaki kodları yazın. (Not: Bu kodlar CubeMX'te tanımlanan **LCD_RS**, **LCD_EN** vb. etiketleri kullanır.)

```

/* lcd.c */
#include "lcd.h"
#include "main.h" // Pin tanımları buradan alınır

// 4-Bit Veri Gönderme Yardımcı Fonksiyonu
void LCD_Send4Bits(uint8_t data) {
    HAL_GPIO_WritePin(LCD_D4_GPIO_Port, LCD_D4_Pin, (data & 1) ? GPIO_PIN_SET :
GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LCD_D5_GPIO_Port, LCD_D5_Pin, ((data >> 1) & 1) ?
GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LCD_D6_GPIO_Port, LCD_D6_Pin, ((data >> 2) & 1) ?
GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LCD_D7_GPIO_Port, LCD_D7_Pin, ((data >> 3) & 1) ?
GPIO_PIN_SET : GPIO_PIN_RESET);
}

// Komut Gönderme
void LCD_SendCommand(uint8_t cmd) {
    HAL_GPIO_WritePin(LCD_RS_GPIO_Port, LCD_RS_Pin, GPIO_PIN_RESET); // RS=0
(Komut)

    LCD_Send4Bits(cmd >> 4); // Üst 4 bit
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_SET);
    HAL_Delay(1);
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_RESET);
    HAL_Delay(1);

    LCD_Send4Bits(cmd & 0x0F); // Alt 4 bit
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_SET);
    HAL_Delay(1);
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_RESET);
    HAL_Delay(2);
}

// Veri Gönderme
void LCD_SendData(uint8_t data) {
    HAL_GPIO_WritePin(LCD_RS_GPIO_Port, LCD_RS_Pin, GPIO_PIN_SET); // RS=1 (Veri)

    LCD_Send4Bits(data >> 4);
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_SET);
    HAL_Delay(1);
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_RESET);
    HAL_Delay(1);

    LCD_Send4Bits(data & 0x0F);
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_SET);
    HAL_Delay(1);
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_RESET);
}

```



```

    HAL_Delay(1);
}

// LCD Başlatma Prosedürü
void LCD_Init(void) {
    HAL_Delay(50);
    LCD_SendCommand(0x02); // 4-bit moda geçiş
    LCD_SendCommand(0x28); // 2 satır, 5x8 font
    LCD_SendCommand(0x0C); // Ekran Açık, İmleç Kapalı
    LCD_SendCommand(0x06); // İmleç sağa kaysın
    LCD_SendCommand(0x01); // Temizle
    HAL_Delay(2);
}

void LCD_Clear(void) {
    LCD_SendCommand(0x01);
    HAL_Delay(2);
}

void LCD_SetCursor(uint8_t row, uint8_t col) {
    uint8_t address;
    if (row == 0) address = 0x80 + col;
    else address = 0xC0 + col;
    LCD_SendCommand(address);
}

void LCD_Print(char *str) {
    while (*str) {
        LCD_SendData(*str++);
    }
}

```

Adım 3: Kütüphanelerin Dahil Edilmesi main.c dosyasının en üst kısmına aşağıdaki kütüphaneler eklenmelidir:

```

/* Includes -----*/
#include "main.h"
#include "lcd.h"    // LCD Fonksiyonları için
#include <stdio.h>  // sprintf fonksiyonu için

```

6.2. Rotary Encoder Algoritması (Kesme Mantığı)

Rotary Encoder (Döner Kodlayıcı) döndürüldüğünde, **CLK (PB13)** pini üzerinde sinyal değişimleri oluşur. Biz bu pini **EXTI (Harici Kesme)** olarak ayarladığımız için, her sinyal değişiminde işlemci ana programı durdurup otomatik olarak HAL_GPIO_EXTI_Callback fonksiyonuna gider.

Yön Tayini Mantığı: Kesme anında (CLK sinyali değiştiğinde), **DT (PB12)** pininin durumu kontrol edilir:

- **Eğer CLK ve DT sinyalleri farklı seviyedeysse: Dönüş Saat Yönünde (CW)** gerçekleşmiştir. Sayaç artırılır.
- **Eğer CLK ve DT sinyalleri aynı seviyedeysse: Dönüş Saat Yönünün Tersine (CCW)** gerçekleşmiştir. Sayaç azaltılır.

6.3. Kaynak Kod (main.c)

Aşağıdaki kod bloklarını main.c dosyası içerisindeki ilgili USER CODE BEGIN alanlarına yerleştiriniz.

1. Global Değişkenlerin Tanımlanması /* USER CODE BEGIN 0 */ bloğuna:

```
/* USER CODE BEGIN 0 */
volatile int16_t encoder_sayac = 0; // Encoder sayacı (Negatif değer alabilir)
char yon_bilgisi[4] = "---"; // Yön metni: "CW " veya "CCW"
char lcd_buffer[16]; // LCD metin tamponu
/* USER CODE END 0 */
```

2. Kesme (Interrupt) Fonksiyonu (Callback) Bu fonksiyon, encoder hareket ettiğinde donanım tarafından **otomatik olarak çağrılır**. Bu kodu main fonksiyonunun dışına, dosyanın en altındaki /* USER CODE BEGIN 4 */ bloğuna yazınız.

```

/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    // Kesme, Encoder'ın CLK pininden mi geldi?
    if(GPIO_Pin == ENC_CLK_Pin)
    {
        // Pinlerin anlık durumunu oku
        // Bu işlem çok hızlı yapılmalıdır (Interrupt içindeyiz)
        GPIO_PinState clk_durumu = HAL_GPIO_ReadPin(ENC_CLK_GPIO_Port,
ENC_CLK_Pin);
        GPIO_PinState dt_durumu = HAL_GPIO_ReadPin(ENC_DT_GPIO_Port, ENC_DT_Pin);

        // Yön Tayini Algoritması
        if (clk_durumu != dt_durumu)
        {
            encoder_sayac++;          // Sağa dönüş (CW) -> Artır
            sprintf(yon_bilgisi, "CW ");
        }
        else
        {
            encoder_sayac--;          // Sola dönüş (CCW) -> Azalt
            sprintf(yon_bilgisi, "CCW");
        }
    }
}
/* USER CODE END 4 */

```

3. Başlangıç ve Ana Döngü main fonksiyonu içerisindeki while(1) döngüsü ve öncesi:

```

/* USER CODE BEGIN 2 */
LCD_Init(); // LCD ekranı başlat

// --- ÖDEV İSTERİ: Açılış Animasyonu ---
LCD_SetCursor(0, 0);
LCD_Print("SCU BİL MUH");
HAL_Delay(2000); // 2 Saniye Bekle
LCD_Clear();

LCD_SetCursor(0, 0);
LCD_Print("DENEY-3 KESME");
LCD_SetCursor(1, 0);
LCD_Print("Grup No: 10");
HAL_Delay(3000); // 3 Saniye Bekle
LCD_Clear();
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1)
{
    // --- 1. Satır: Sayaç Değeri ---
    // Not: Sayacı okumak için polling yapmıyoruz,
    // kesme fonksiyonu arka planda değişkeni güncelliyor.
    LCD_SetCursor(0, 0);
    sprintf(lcd_buffer, "ENC Val: %4d", encoder_sayac);
    LCD_Print(lcd_buffer);

    // --- 2. Satır: Yön Bilgisi (SW6 Kontrolü) ---
    // SW6 (PD10) anahtarı açık mı? (SET mi?)
    if (HAL_GPIO_ReadPin(SW6_GPIO_Port, SW6_Pin) == GPIO_PIN_SET)
    {
        LCD_SetCursor(1, 0);
        sprintf(lcd_buffer, "ENC Dir: %s", yon_bilgisi);
        LCD_Print(lcd_buffer);
    }
    else
    {
        // Anahtar kapalıysa satırını temizle
        LCD_SetCursor(1, 0);
        LCD_Print("                ");
    }

    HAL_Delay(50); // Ekran tazeleme hızı (50ms yeterlidir)
}
/* USER CODE END WHILE */

```

7. Pratik Uygulama ve Test Adımları

Yazdığımız kodun STM32 kartına yüklenmesi ve deney seti üzerinde test edilmesi için aşağıdaki adımları izleyin.

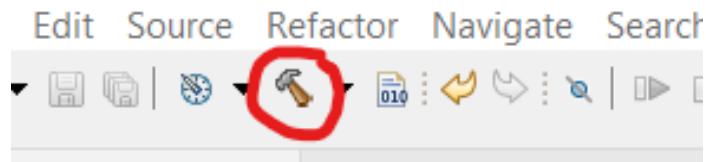
Adım 1: Fiziksel Bağlantı

1. ArmApp18 deney seti üzerindeki **STM32F4 Discovery** kartının üst kısmında bulunan **mini-USB (USB ST-LINK)** girişine kabloyu takın.
2. Kablonun diğer ucunu bilgisayarınızın USB portuna bağlayın.
3. Kart üzerindeki **PWR** (Güç) LED'inin yandığını ve bilgisayarınızın donanımı tanıdığını doğrulayın.



Adım 2: Derleme ve Yükleme (Build & Run)

1. **Derleme:** STM32CubeIDE üst menüsündeki **"Çekiç" (Build)** ikonuna tıklayın (veya Ctrl+B).



- Console penceresinde "0 errors, 0 warnings" mesajını görmelisiniz.

8. Sonuç ve Değerlendirme

Bu laboratuvar çalışmasında, **STM32F407** mikrodenetleyicisinin **EXTI (External Interrupt)** birimi kullanılarak gerçek zamanlı bir sensör okuma uygulaması gerçekleştirilmiştir.

Elde Edilen Sonuçlar:

1. **Kesme Mekanizması:** Rotary Encoder'dan gelen sinyaller, işlemciyi sürekli meşgul eden "Polling" yöntemi yerine, sadece değişim anında tetiklenen "Interrupt" yöntemiyle yakalanmıştır. Bu sayede işlemci kaynakları verimli kullanılmıştır.
2. **Yön ve Konum Tespiti:** Encoder'ın CLK ve DT pinleri arasındaki faz farkı analiz edilerek dönüş yönü (CW/CCW) ve adım sayısı hatasız bir şekilde tespit edilmiştir.
3. **Kullanıcı Arayüzü:** LCD ekran üzerinden anlık sayaç değeri gösterilmiş ve **SW6** anahtarı ile kullanıcıya dinamik bilgi (Yön Bilgisi) sunulmuştur.

Bu deney ile gömülü sistemlerde asenkron olayların yönetimi ve donanım kesmelerinin önemi kavranmıştır.



